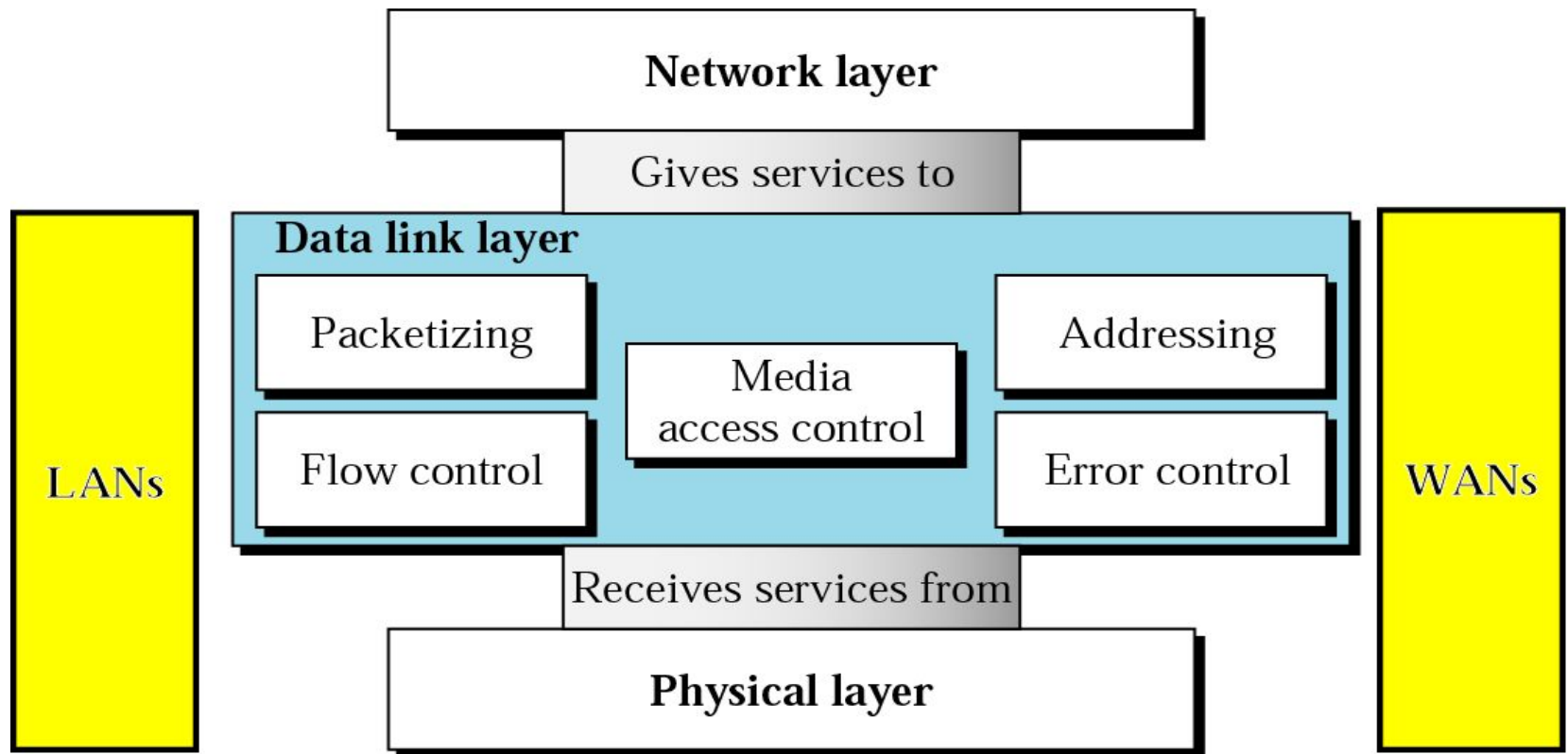


Computer Communication & Networks

Lecture 11

Datalink Layer: Error Detection

Data Link Layer



Data Link Layer Topics to Cover

Error Detection and Correction

Data Link Control and Protocols

Multiple Access

Local Area Networks

Wireless LANs

Error Detection

- Why we need it ?
 - To **avoid retransmission** of whole packet or message
- What to do if error detected ?
 - **Discard**, and request a new copy of the frame:
 - explicitly or implicitly
 - Try to **correct** error, if possible

Note

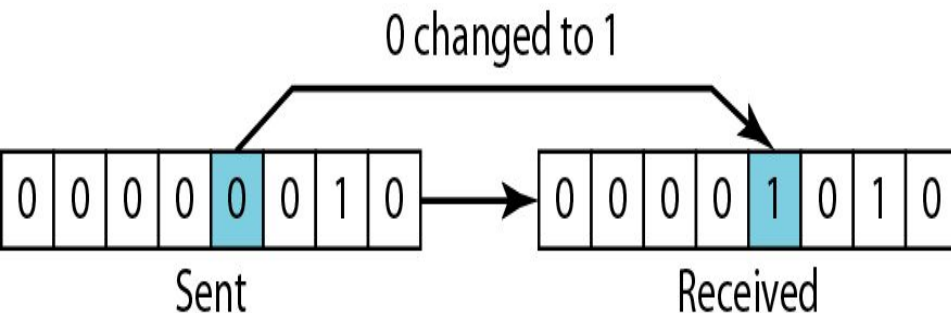
**Data can be corrupted
during transmission.**

**Some applications require that
errors be detected and corrected.**

Types of Errors

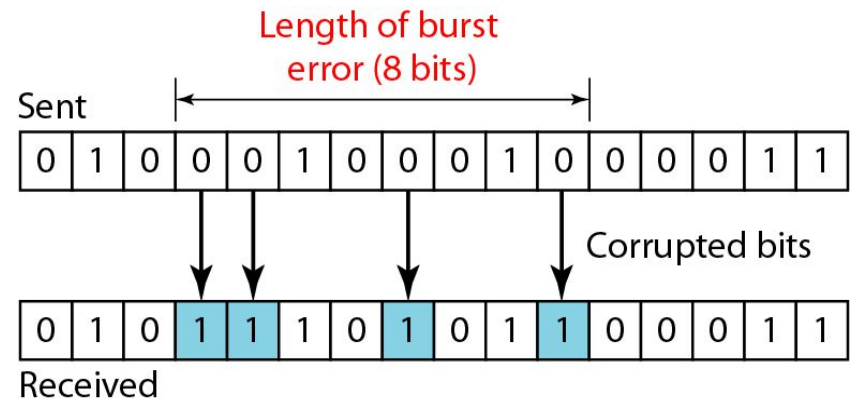
■ Single Bit Error

- In a single-bit error, only 1 bit in the data unit has changed.



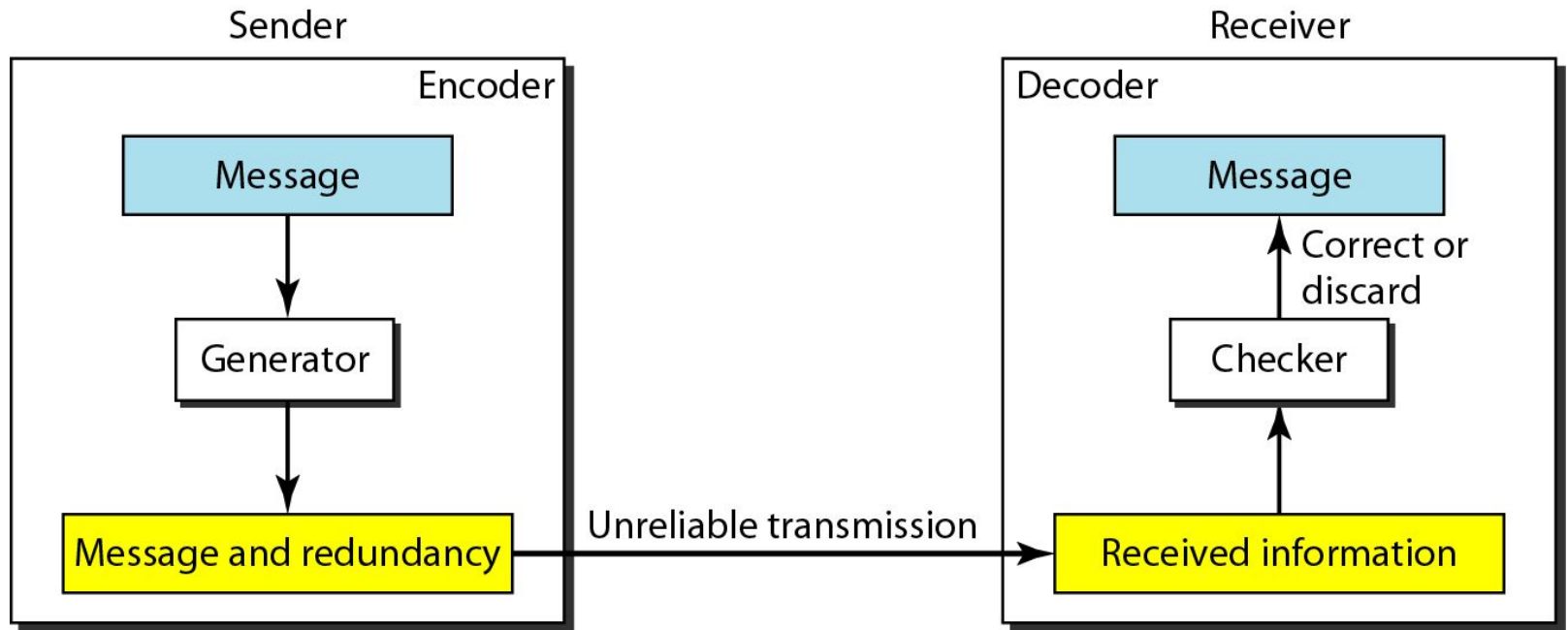
■ Burst Error

- A burst error means that 2 or more bits in the data unit have changed.

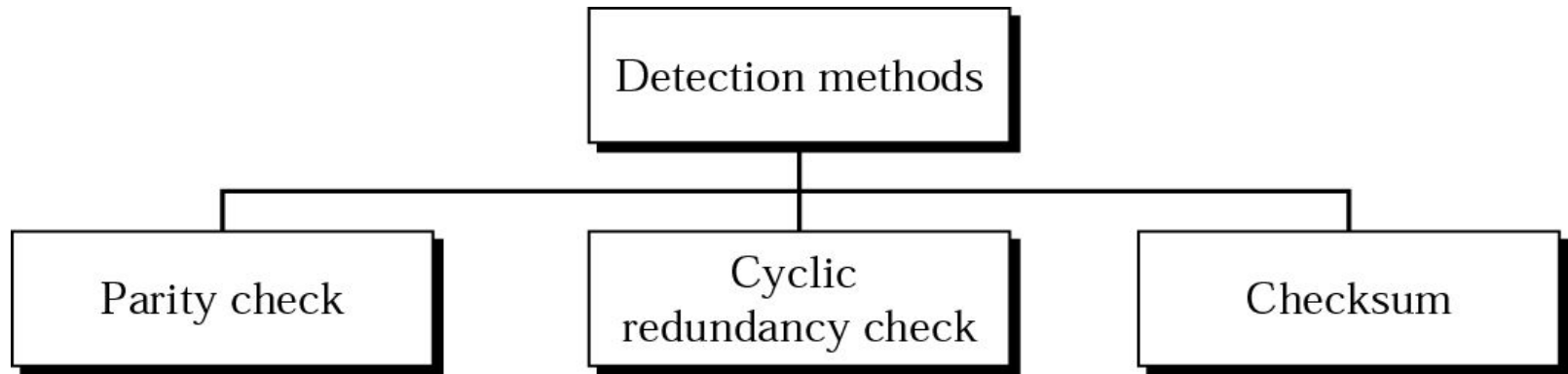


Redundancy

- To detect or correct errors, we need to send extra (redundant) bits with data.

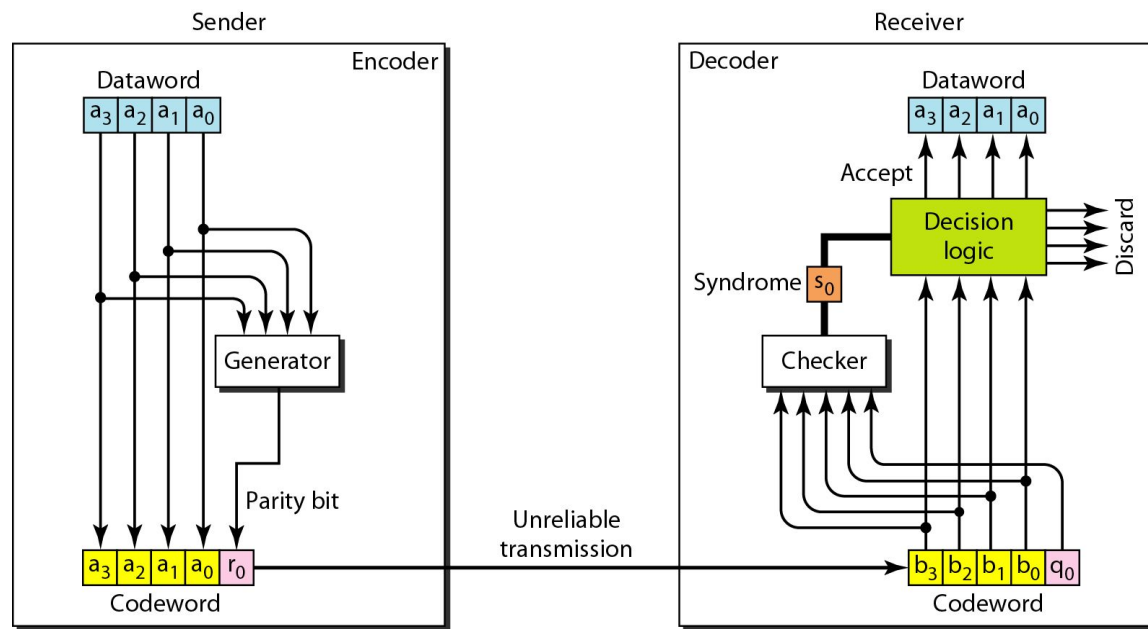


Error Detection



Simple Parity Check

- A simple parity-check code is a single-bit error-detecting code in which $n = k + 1$.



Example

- Let us look at some transmission scenarios. Assume the sender sends the dataword 1011. The codeword created from this dataword is 10111, which is sent to the receiver. We examine five cases:
 1. No error occurs; the received codeword is 10111. The syndrome is 0. The dataword 1011 is created.
 2. One single-bit error changes a_1 . The received codeword is 10011. The syndrome is 1. No dataword is created.
 3. One single-bit error changes r_0 . The received codeword is 10110. The syndrome is 1. No dataword is created.

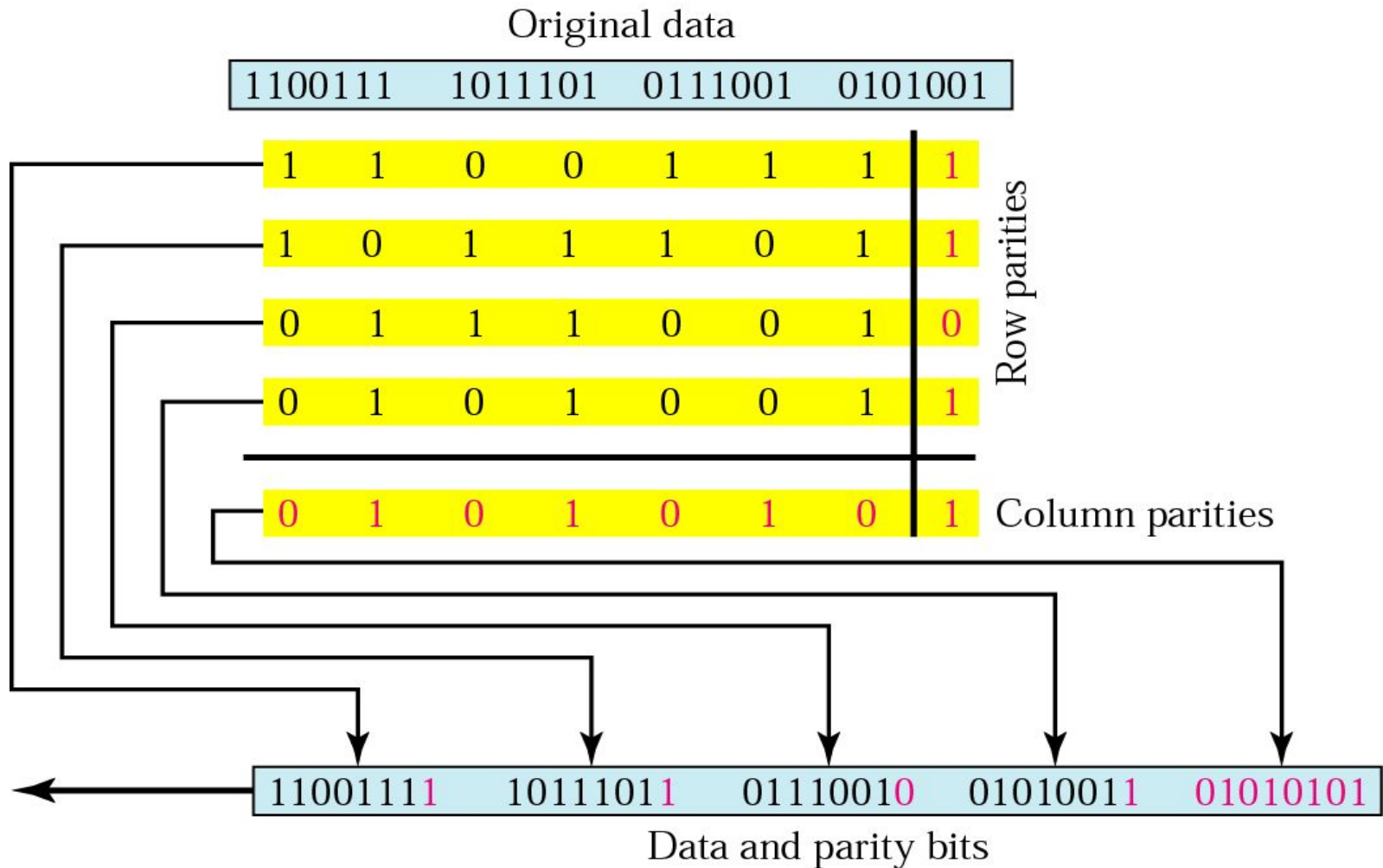
Example (contd.)

4. An error changes r_0 and a second error changes a_3 .
The received codeword is 00110. The syndrome is 0. The dataword 0011 is created at the receiver. Note that here the dataword is wrongly created due to the syndrome value.
5. Three bits— a_3 , a_2 , and a_1 —are changed by errors.
The received codeword is 01011. The syndrome is 1. The dataword is not created. This shows that the simple parity check, guaranteed to detect one single error, can also find any odd number of errors.

Performance

- A Simple parity check can detect all single-bit errors. It can detect burst errors only if the total number of errors in each data unit is odd.

Two-dimensional Parity-check Code



Two-Dimensional Parity

0	1	0	0	0	1	1	1
0	1	1	0	0	0	1	1
0	1	1	0	1	1	1	1
0	1	1	0	0	0	0	0
1	0	0	1	0	0	1	1
0	1	1	0	1	1	0	0
1	1	0	1	0	1	0	0

Example

- Suppose the following block is sent:

10101001 00111001 11011101 11100111 10101010

- However, it is hit by a burst noise of length 8, and some bits are corrupted.

10100011 10001001 11011101 11100111 10101010

- When the receiver checks the parity bits, some of the bits do not follow the even-parity rule and the whole block is discarded.

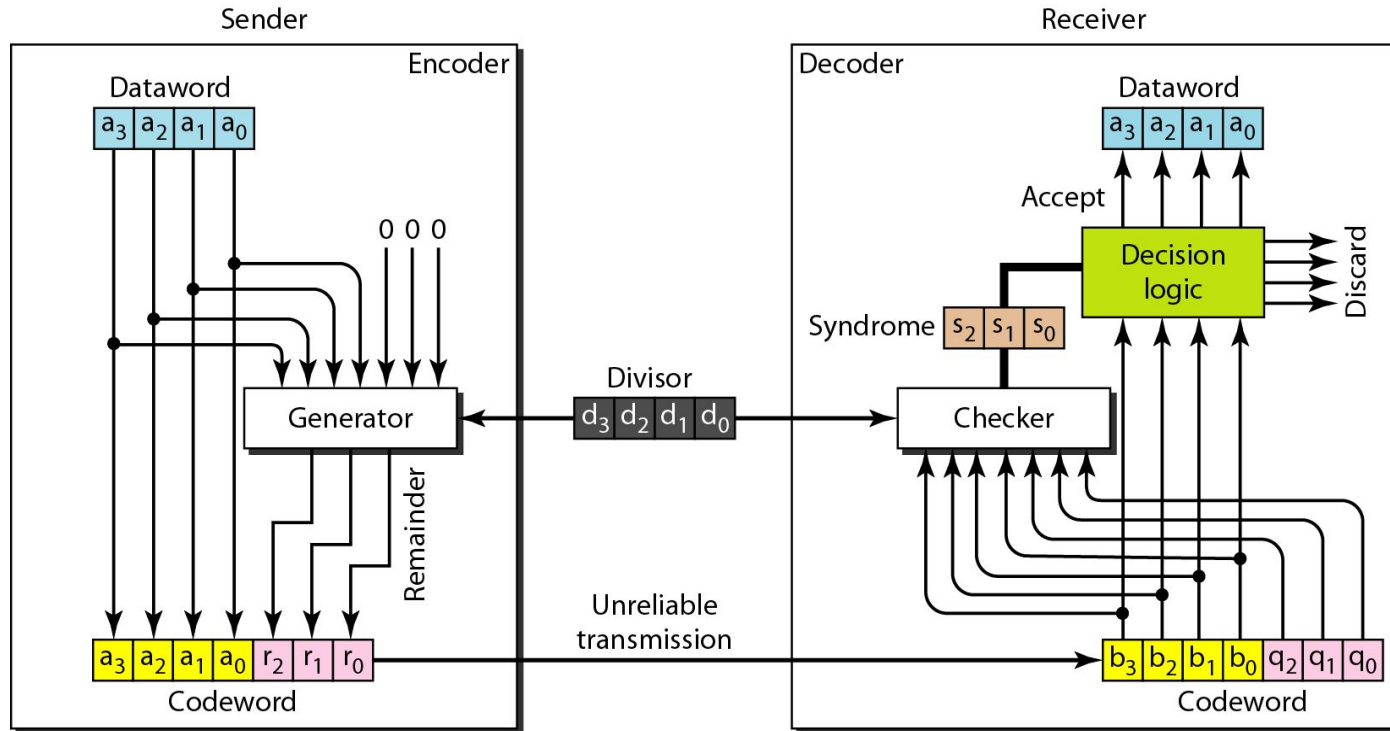
10100011 10001001 11011101 11100111 10101010

Performance

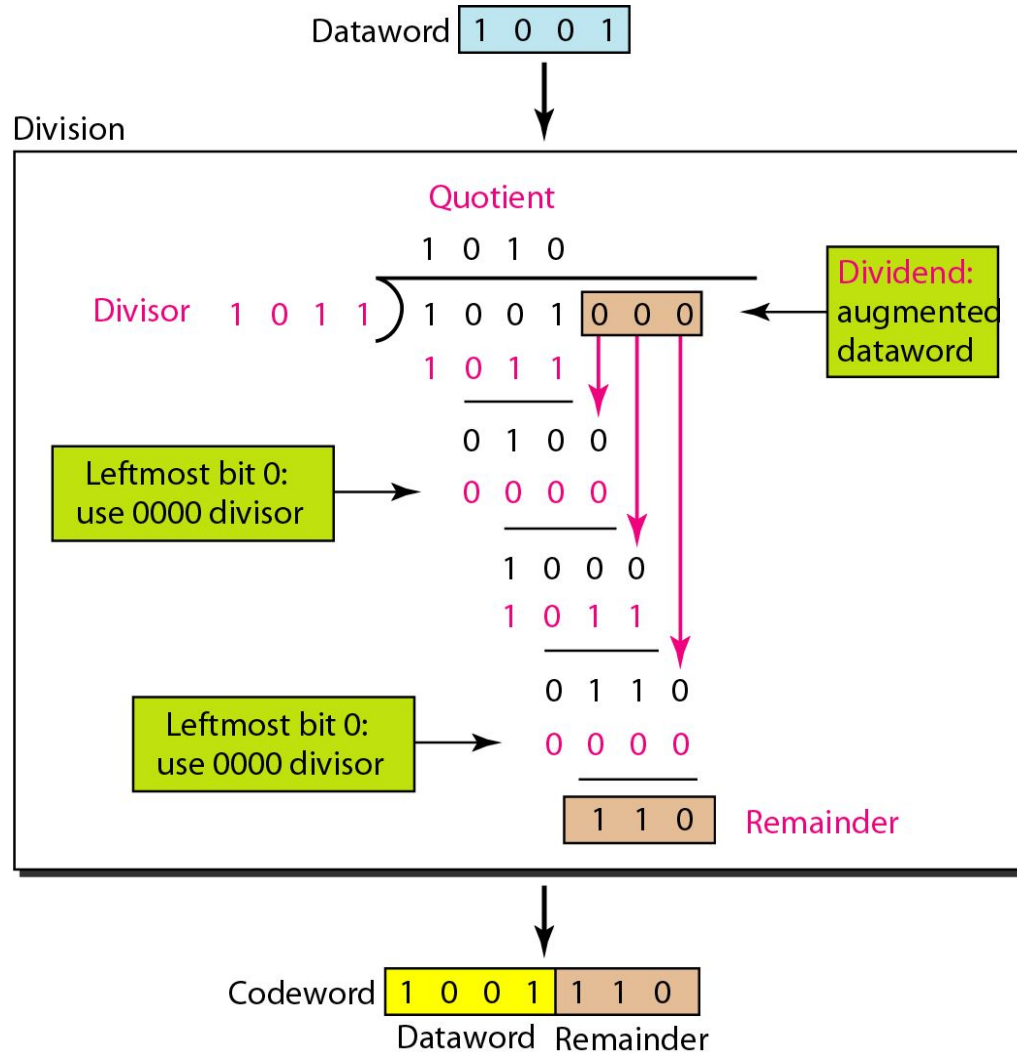
- 2D parity check increases the likelihood of detecting burst errors. As we have seen in the example given in the previous slide, a redundancy of n bits can easily detect a burst error of n bits. There is, however, one pattern of errors that remains un-detectable. If 2 bits in one data unit are damaged and 2 bits in exactly in the same positions in another data unit are also damaged, the checker will not detect an error.
-

Cyclic Redundancy Check

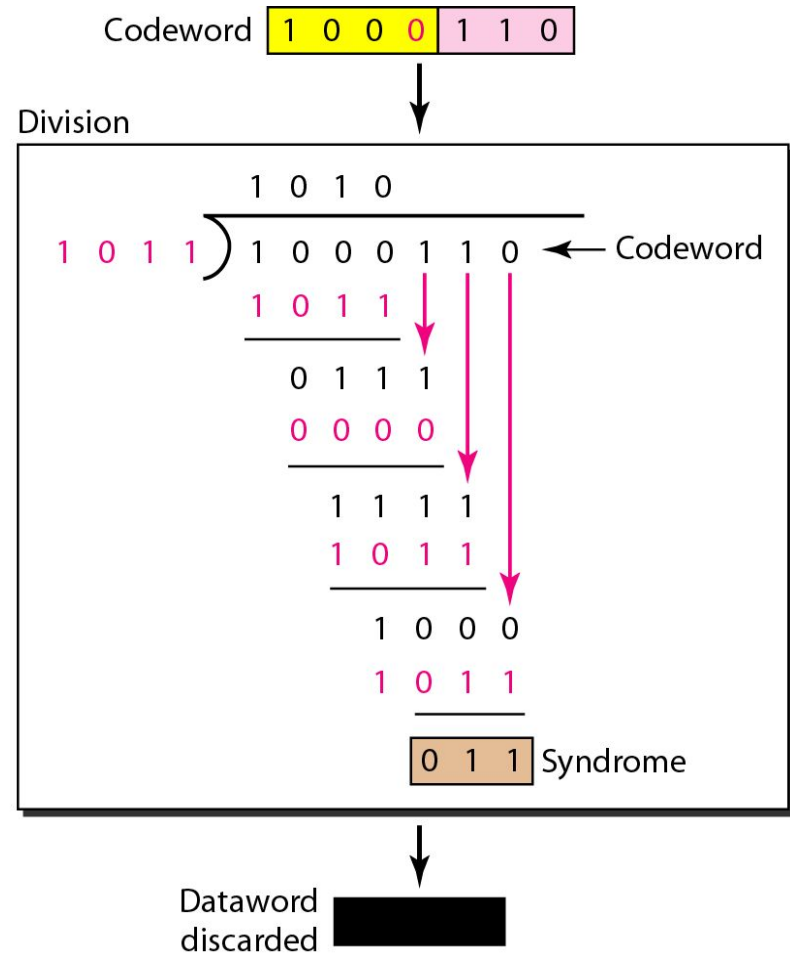
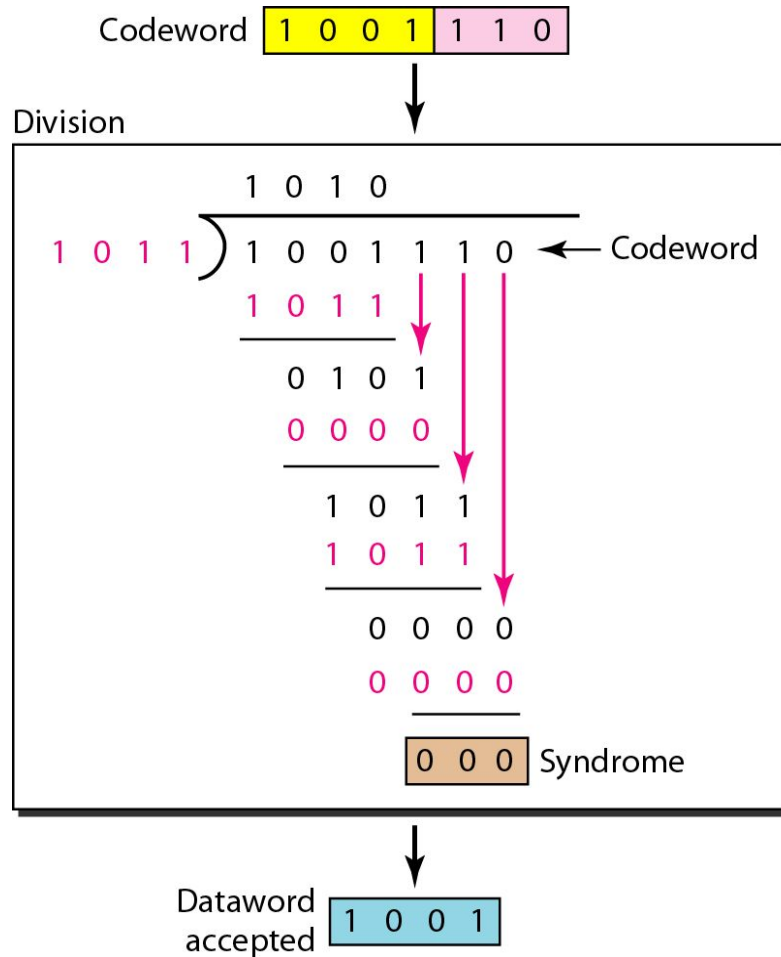
- Cyclic Redundancy Check (also called polynomial code)
- Based on modulo-2 binary division
 - No carries (because it's modulo-2)
 - Subtraction is equivalent to XOR



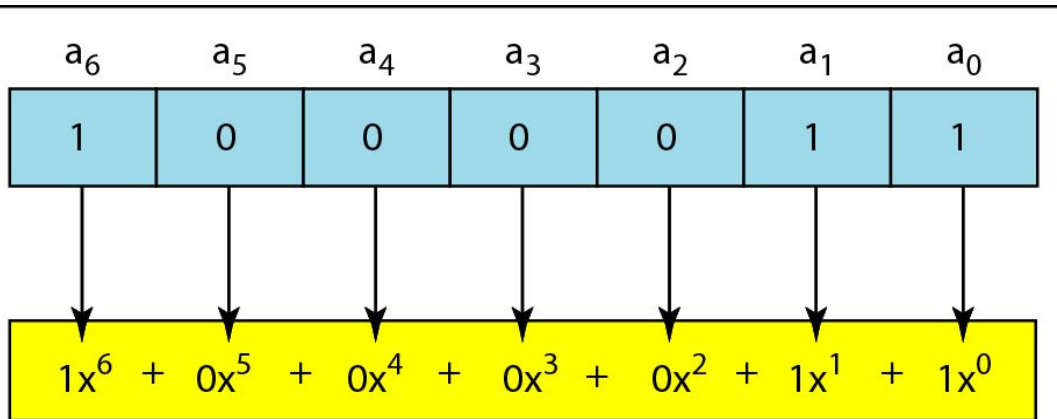
Division in CRC encoder



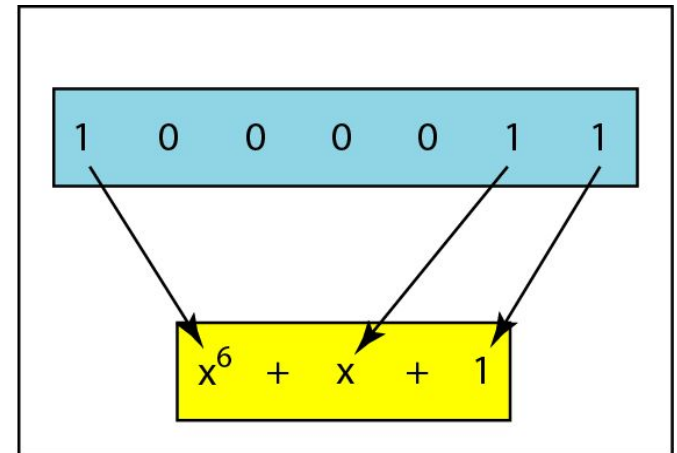
Division in the CRC decoder for two cases



A polynomial to represent a binary word

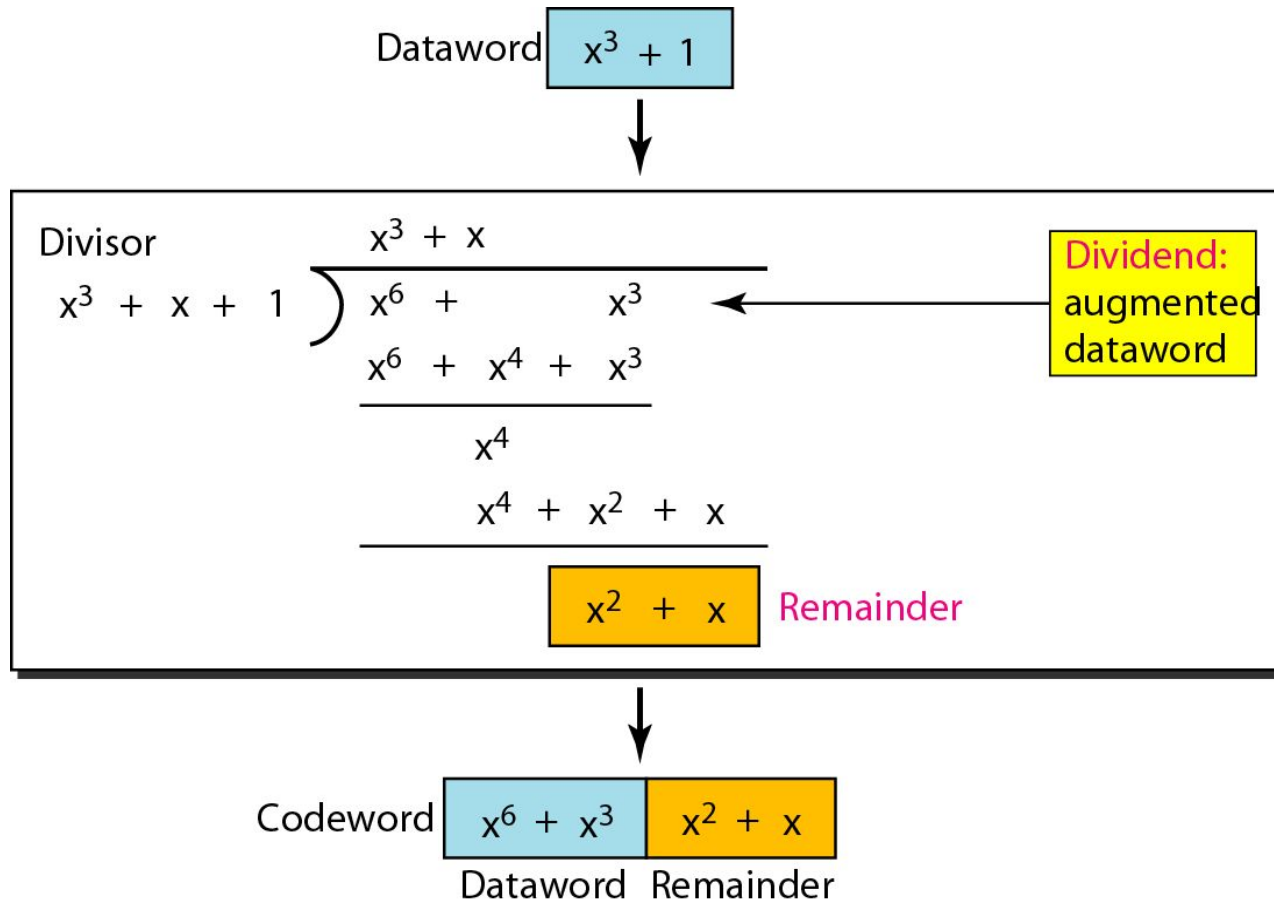


a. Binary pattern and polynomial



b. Short form

CRC division using polynomials



Performance

- In a cyclic code, those $e(x)$ errors that are divisible by $g(x)$ are not caught.
- If the generator has more than one term and the coefficient of x^0 is 1, all single errors can be caught.
- A generator that contains a factor of $x + 1$ can detect all odd-numbered errors.

Performance (contd.)

- All burst errors with $L \leq r$ will be detected.
- All burst errors with $L = r + 1$ will be detected with probability $1 - (1/2)^{r-1}$.
- All burst errors with $L > r + 1$ will be detected with probability $1 - (1/2)^r$.

Example

Which of the following $g(x)$ values guarantees that a single-bit error is caught? For each case, what is the error that cannot be caught?

- a. $x + 1$ b. x^3 c. 1*

Solution

- a. No x^i can be divisible by $x + 1$. Any single-bit error can be caught.*
- b. If i is equal to or greater than 3, x^i is divisible by $g(x)$. All single-bit errors in positions 1 to 3 are caught.*
- c. All values of i make x^i divisible by $g(x)$. No single-bit error can be caught. This $g(x)$ is useless.*
-

Properties of Good Polynomials

- A good polynomial generator needs to have the following characteristics:
- It should have at least two terms.
- The coefficient of the term x^0 should be 1.
- It should have the factor $x + 1$.

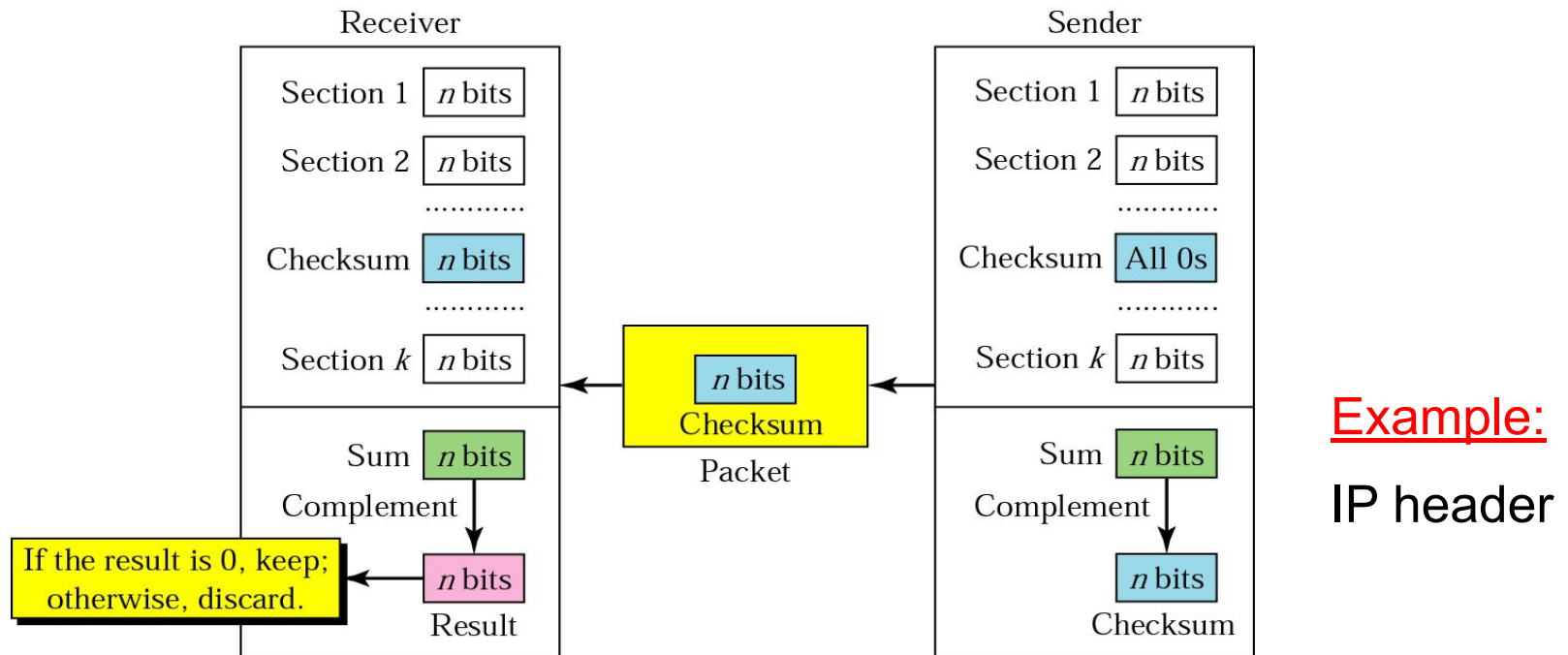
Table 10.7 *Standard polynomials*

<i>Name</i>	<i>Polynomial</i>	<i>Application</i>
CRC-8	$x^8 + x^2 + x + 1$	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$	ATM AAL
CRC-16	$x^{16} + x^{12} + x^5 + 1$	HDLC
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	LANs

Summary

- CRC can detect all burst errors that affect an odd number of bits.
- CRC can detect all burst errors of length less than or equal to the degree of polynomial.
- CRC can detect, with a very high probability, burst errors of length greater than the degree of polynomial.

Checksum



1. The data unit is divided into k sections, each of n bits
2. All sections are added using 1's complement
3. The sum is complemented
4. The checksum is sent with data

Checksum: Sending

Suppose the following block of 16 bits is to be sent using a checksum of 8 bits.

10101001 00111001

The numbers are added using one's complement

$$\begin{array}{r} 10101001 \\ 00111001 \\ \hline \end{array}$$

Sum 11100010

- Checksum **00011101** (Take 1's complement of Sum)
- The pattern sent is 10101001 00111001 **00011101**

Checksum: Receiving

Now suppose the receiver receives the pattern sent and there is no error.

10101001 00111001 00011101

- When the receiver adds the three sections, it will get all 1s, which, after complementing, is all 0s and shows that there is no error.

10101001

00111001

00011101

Sum 11111111

Complement 00000000 means that the pattern is OK.

Checksum: Error

A burst error of length 5 that affects 4 bits.

10101111 111111001 00011101

- When the receiver adds the three sections, it gets

10101111

11111001

00011101

Partial Sum **1** 11000101

Carry **1**

Sum 11000110 => Complement

00111001 !!!??

Readings

- Chapter 10 (B.A Forouzan)
 - Section 10.1, 10.3, 10.4
 - (Cover only those contents which are related to topics covered in class)
-

